

Consumo de servicios web con REST API

Modalidad: Práctica de laboratorio, individual

1. Propósito de la actividad

Que el estudiante consuma servicios web a través de una REST API, primero con **Postman** y después con **Python** (lenguaje multiparadigma), interpretando respuestas en formato **JSON** y comparándolas con **XML**, para reafirmar los temas de la unidad:

- APIs y análisis de JSON.
- REST API utilizando Postman.
- REST API utilizando un lenguaje multiparadigma.
- Servicios web con API REST, JSON o XML.

2. Materiales

- Equipo de cómputo con acceso a Internet.
- Postman (aplicación de escritorio o web).
- Python 3 y un IDE (VS Code, PyCharm u otro).
- Librería requests (instalar con `pip install requests`).
- APIs públicas que **no requieren registro ni llave**:
 - o JSONPlaceholder: `https://jsonplaceholder.typicode.com` (API de práctica con recursos `posts`, `users`, `comments`, etc.). o Open-Meteo: `https://api.open-meteo.com/v1/forecast` (pronóstico del clima).

*Nota: JSONPlaceholder **simula** las operaciones de escritura. Responde como si el POST, PUT o DELETE hubieran funcionado, pero los datos no se guardan realmente. Es normal que un GET posterior no muestre el cambio.*

3. Parte A. Repaso conceptual (20 min)

Responde en tu cuaderno o en un documento, con tus propias palabras:

1. ¿Qué es una API y qué significa que una API sea REST?
api: reglas que permiten la comunicación e intercambio de datos entre software.
rest: estilo de arquitectura basado en http, comunicación sin estado (stateless) y uris para manipular recursos (ej. json).
2. Describe la diferencia entre cliente y servidor en una petición HTTP.
Cliente: Quien inicia la comunicación enviando una petición
Servidor: Quien procesa la petición y la responde con datos o estado

3. Completa la tabla de métodos HTTP:

Método	Operación CRUD	Para qué se usa	Código de éxito típico
GET	READ	Consultar recursos	200
POST	CREATE	Crear recursos	201
PUT	UPDATE	Reemplazar completamente un recurso	200/
PATCH	UPDATE	Modificar parcial mente un recurso	200
DELETE	DELETE	Eliminar un recurso	200/204

4. Explica qué significan los códigos de estado: **200, 201, 204, 400, 401, 404 y 500**.
 - 200 (OK): Petición exitosa.
 - 201 (Created): Recurso creado con éxito.
 - 204 (No Content): Éxito sin datos en el cuerpo.
 - 400 (Bad Request): Error de sintaxis en la petición.
 - 401 (Unauthorized): Requiere autenticación.
 - 404 (Not Found): Recurso no encontrado.
 - 500 (Internal Server Error): Error en el servidor.
5. Escribe un objeto JSON que represente a un dispositivo de red con: nombre, dirección IP, modelo, estado (activo o inactivo) y una lista de interfaces. Indica qué tipo de dato es cada valor.

Inventario = {

"nombre": "Router-Core",

"direccion_ip": "192.168.1.1",

```

"modelo": "Cisco 4331",

"estado": true,

"interfaces": ["Gi0/0/0", "Gi0/0/1"]

}

```

4. Parte B. Consumo de la API con Postman (60 min)

Crea una **colección** en Postman llamada `Unidad2_Act2_TuNombre` y dentro de ella una petición por cada ejercicio. Toma captura de pantalla de la petición y de la respuesta (cuerpo, código de estado y encabezados).

#	Petición	URL	Qué debes observar
B1	GET lista de publicaciones	https://jsonplaceholder.typicode.com/posts	Código 200 y que el cuerpo sea una lista (arreglo) de objetos.
B2	GET un recurso	https://jsonplaceholder.typicode.com/posts/1	Campos <code>userId</code> , <code>id</code> , <code>title</code> , <code>body</code> .
B3	GET con parámetros	https://jsonplaceholder.typicode.com/posts?userId=3	Solo regresan publicaciones del usuario 3. Anota cuántas son.
B4	GET recurso anidado	https://jsonplaceholder.typicode.com/posts/1/comments	Comentarios de la publicación 1.
B5	POST crear	https://jsonplaceholder.typicode.com/posts	En Body → raw → JSON envía un objeto con <code>title</code> , <code>body</code> y <code>userId</code> . Código 201 y un id nuevo.
B6	PUT reemplazar	https://jsonplaceholder.typicode.com/posts/1	Envía un objeto completo. Compara con B2.
B7	PATCH modificar	https://jsonplaceholder.typicode.com/posts/1	Envía solo <code>{"title": "Nuevo título"}</code> .
B8	DELETE eliminar	https://jsonplaceholder.typicode.com/posts/1	Código 200 y cuerpo vacío <code>{}</code> .
B9	GET recurso inexistente	https://jsonplaceholder.typicode.com/posts/9999	Código 404. Explica qué significa.
B10	GET con parámetros de consulta	https://api.open-meteo.com/v1/forecast?latitude=20.5981&longitude=100.398&current=temperature_2m,wind_speed_10m&current=temperature_2m,wind_speed_10m	Temperatura y viento actuales de Querétaro. Identifica las unidades en el campo <code>current_units</code> .

B1



B2



B3

Regresa 9 publicacione del usuario



B4

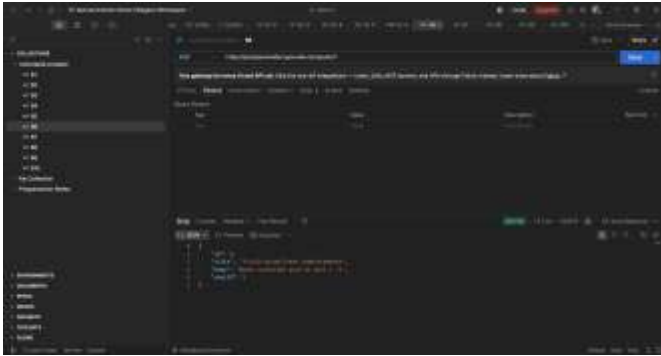
Regresa los comentarios del Id 1



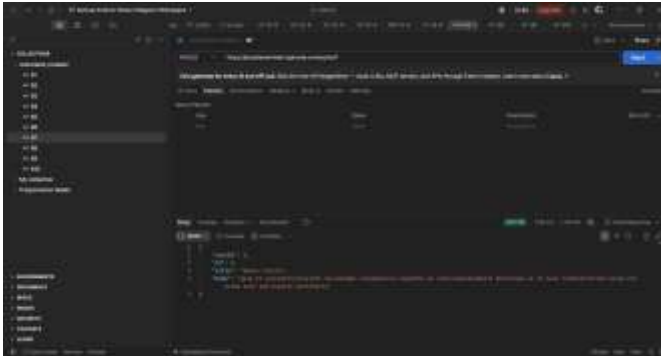
B5



B6



B7

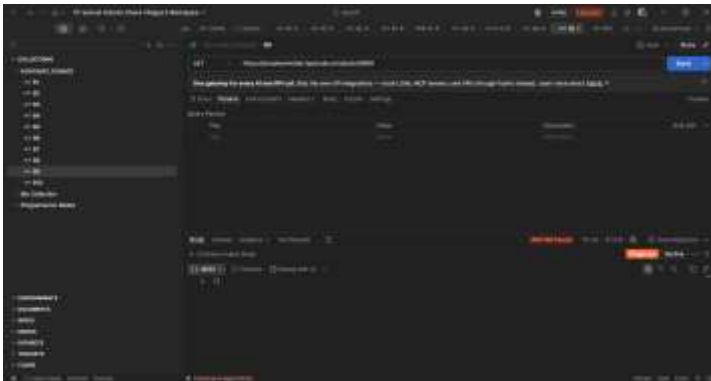


B8



B9

Indica que no existe por que el servidor solo tiene del 1 al 100 y el 9999 no existe



B10



Preguntas de la Parte B

1. ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía?
PUT envía el objeto completo para reemplazarlo; PATCH solo envía los campos a modificar.
2. ¿Qué encabezado Content-Type regresa el servidor y qué indica?
Devuelve application/json; charset=utf-8, e indica que la respuesta está en formato JSON y codificada en UTF-8
3. 10, ¿qué parte de la URL es la ruta y cuál son los parámetros de consulta (query params)?

En Componentes de la URL en B10:

Ruta: /v1/forecast

Query Params: latitude=20.5981, longitude=-100.398 y
current=temperature_2m,wind_speed_10m

5. Parte C. Consumo de la API con Python

Crea una carpeta `actividad_rest` y un archivo `cliente_rest.py` por cada ejercicio

C1. Primer GET

```
import requests

BASE = "https://jsonplaceholder.typicode.com"

respuesta = requests.get(f"{BASE}/posts/1", timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipo de contenido:", respuesta.headers["Content-Type"])

publicacion = respuesta.json() # convierte el JSON en un diccionario de Python
print("Título:", publicacion["title"])
```

C2. Recorrer una lista y usar parámetros

```
respuesta = requests.get(f"{BASE}/posts", params={"userId": 3}, timeout=10)
publicaciones = respuesta.json() # lista de diccionarios
print("Total:", len(publicaciones))
for p in publicaciones:
    print(f"{p['id']}>3 | {p['title']}")
```

C3. POST, PUT y DELETE

```
nuevo = {"title": "Prueba de red", "body": "Contenido de ejemplo", "userId": 1}
r = requests.post(f"{BASE}/posts", json=nuevo,
timeout=10) print("POST:", r.status_code, r.json())

r = requests.put(f"{BASE}/posts/1", json=**nuevo, "id": 1}, timeout=10)
print("PUT:", r.status_code, r.json())

r = requests.delete(f"{BASE}/posts/1", timeout=10) print("DELETE:",
r.status_code)
```

C4. Manejo de errores

Modifica C1 para que consulte `/posts/9999` y maneje el error correctamente:

```
try:
    r = requests.get(f"{BASE}/posts/9999", timeout=10)
    r.raise_for_status() # lanza excepción si el código es 4xx o 5xx
print(r.json()) except requests.exceptions.HTTPError as e:    print("Error HTTP:",
e) except requests.exceptions.RequestException as e:    print("Error de conexión:",
e)
```

C5. Consumir otra API (clima)

Escribe una función `obtener_clima(latitud, longitud)` que consulte Open-Meteo y regrese un diccionario con la temperatura y la velocidad del viento. Usa `params` en lugar de armar la URL a mano.

```
def obtener_clima(latitud, longitud):    url =
"https://api.open-meteo.com/v1/forecast"
parametros = {
    "latitude": latitud,
    "longitude": longitud,
    "current": "temperature_2m,wind_speed_10m",
}
r = requests.get(url, params=parametros, timeout=10)
r.raise_for_status()
return r.json()["current"]
```

Probarla con tres ciudades distintas (por ejemplo Querétaro: 20.59, -100.39) y mostrar los resultados en una tabla impresa en consola.

6. Parte D. JSON frente a XML (30 min)

1. Toma la respuesta JSON de `/posts/1` y escribe a mano su equivalente en XML.
2. Con Python, convierte el diccionario a XML usando `xml.etree.ElementTree`:

```
import xml.etree.ElementTree as ET
raiz = ET.Element("publicacion")
for clave, valor in publicacion.items():
    ET.SubElement(raiz, clave).text = str(valor)

print(ET.tostring(raiz, encoding="unicode"))
```

3. Completa la tabla comparativa:

Criterio	JSON	XML
Legibilidad		
Tamaño del mensaje		
Tipos de datos		
Uso común en APIs REST		

7. Parte E. Mini proyecto integrador (60 min)

Desarrolla un script `reporte_api.py` que:

1. Consulte `/users` de JSONPlaceholder y obtenga el nombre, correo y ciudad (`address.city`) de cada usuario.
2. Para cada usuario, consulte cuántas publicaciones tiene (`/posts?userId=<id>`).
3. Consulte el clima actual de un punto geográfico de tu elección.
4. Maneje errores de conexión y de código de estado.
5. Guarde el resultado en un archivo `reporte.json` (usa `json.dump` con `indent=4` y `ensure_ascii=False`) y también imprima un resumen en pantalla.

Estructura sugerida del archivo de salida:

```
{
  "clima": {"temperature_2m": 0.0, "wind_speed_10m": 0.0},
  "usuarios": [
    {"nombre": "...", "correo": "...", "ciudad": "...", "publicaciones": 0}
  ]
}
```

8. Entregables (portafolio de evidencias)

1. Documento con las respuestas de la Parte A, las preguntas de la Parte B y la tabla de la Parte D.
2. Exportación de la colección de Postman (archivo `.json`) y capturas de pantalla de B1 a B10.
3. Archivos `.py` de las Partes C y E, con comentarios.
4. Archivo `reporte.json` generado por el mini proyecto.
5. Conclusión breve (5 a 8 líneas): ¿en qué casos de administración de redes serviría consumir una API REST?

9. Lista de verificación para la evaluación

Criterio	Cumple	No cumple
Identifica correctamente métodos HTTP y códigos de estado		
Ejecuta las 10 peticiones en Postman con evidencia		
Envía cuerpo JSON y parámetros de consulta correctamente		
Consume la API con Python usando <code>requests</code>		
Convierte respuestas JSON a estructuras de Python (diccionarios y listas)		
Criterio	Cumple	No cumple
Maneja errores y tiempos de espera (<code>timeout</code> , <code>raise_for_status</code>)		
Compara JSON y XML con un ejemplo funcional		

El mini proyecto genera el archivo reporte.json correcto		
Código ordenado, comentado y entregado a tiempo		